

State of ML Reconstruction Framework

Jack Carlton
University of Kentucky

Principles to Keep Workload Scalable

I choose to break down scalability into three categories:

1. Data

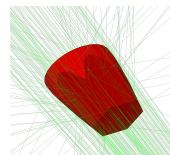
- a. ML pipeline does not change behavior as data set grows
 - i. Still scales in execution time

2. Compute

- a. ML pipeline does not change behavior as compute power is changed
 - i. Still scales in execution time
 - ii. Ex. Pipeline runs on developer's laptop or a CPU/GPU cluster

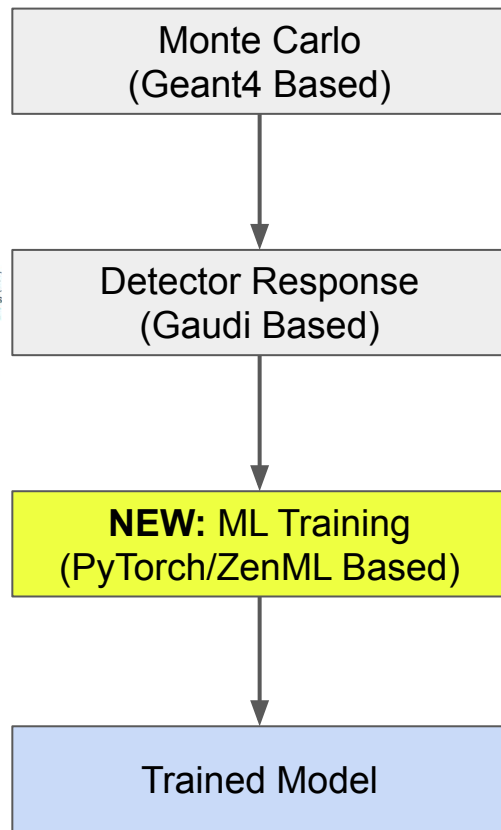
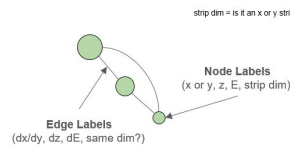
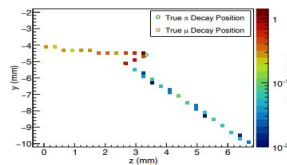
3. Codebase

- a. ML pipeline does not (greatly) change behavior as complexity grows
 - i. New models, stages, or data products require additional code, not (major) code rewrites
 - ii. Iteration speed does not (greatly) degrade with system size (i.e. keep things modular)



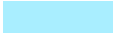
Simulation Integration for Training

- New [DetResponse](#) sub algorithm outputs ML ready [parquet](#) files
 - Currently only extracts ATAR data
 - Additional Gaudi tool “stages” can extend the dataset
 - Setup is currently messy, [guide available](#)
 - Setup should be simpler once integrated with v3
- [PioneerML](#) streams parquet data to models and manages training
 - Outputs a trained model to be used for inference in full reconstruction chain

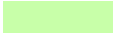


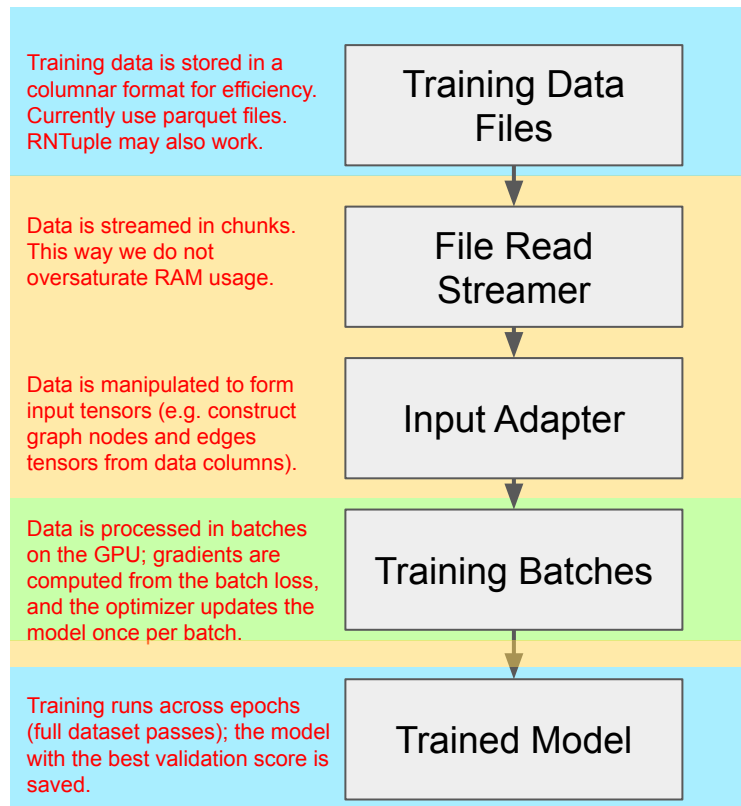
Training Data Flow

- Training data is **streamed** into RAM
 - Streamer reads one chunk of events at a time
- Input construction is vectorized over the chunk
 - NumPy/PyTorch efficiently build tensors (e.g. nodes, edges) using precompiled C++
 - For fully connected graphs, input sizes grow $O(N^2)$ as edges dominate
 - Deriving graphs is almost always faster than storing them
- For large models, training time often dominates per batch
 - Graph transformers tend to be large
- Streaming is slower than loading all data at once, but scales to arbitrarily large datasets

 = Data in storage (SSD(s) or HDD(s))

 = Data in system(s) RAM (CPU Tasks)

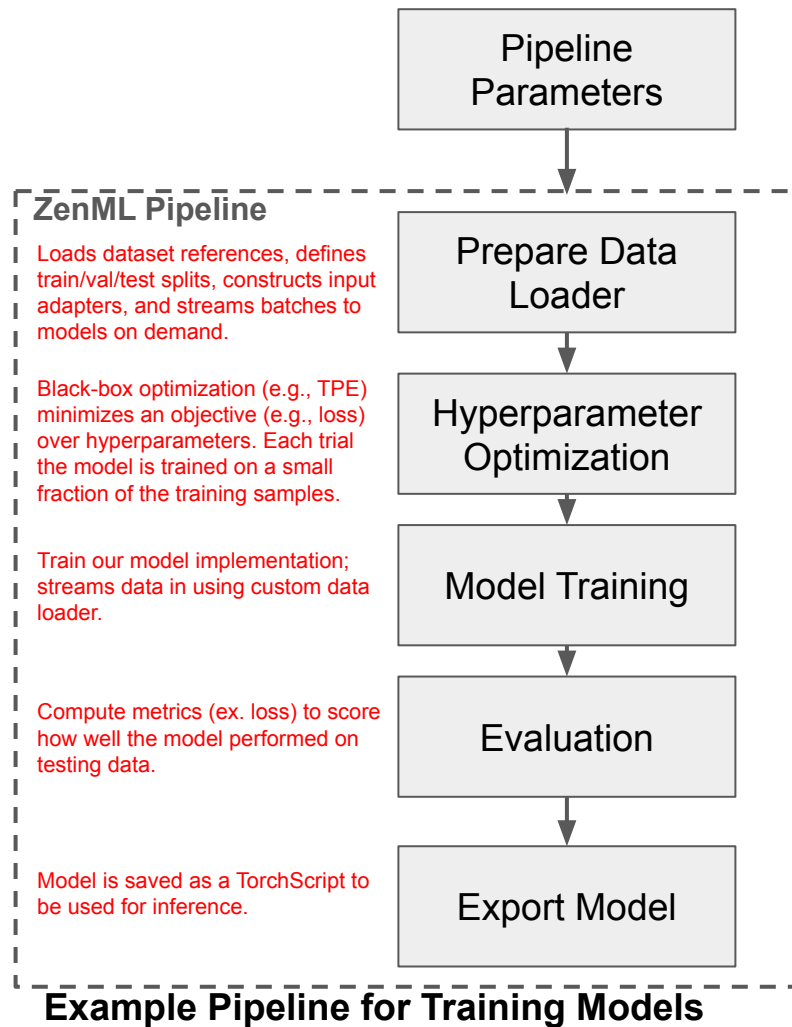
 = Data in GPU(s) VRAM (GPU Tasks)

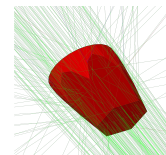


Data Flow for Training Models

Model Training Pipeline

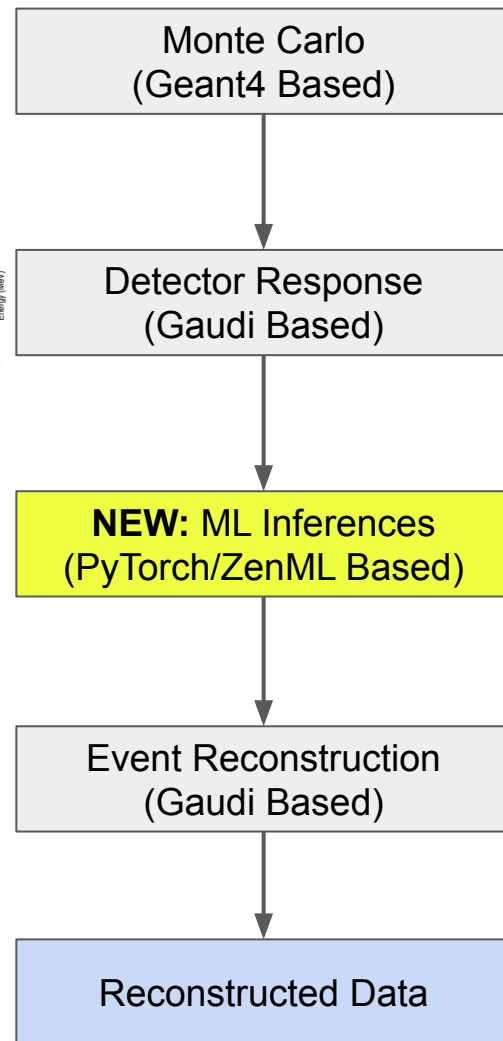
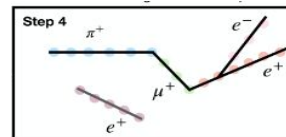
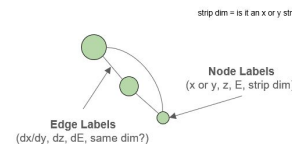
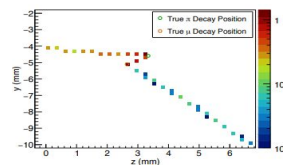
- [PyTorch](#)
 - Provides fast tensor operations for vectorized computation
 - Avoids large python loops when making graphs from feature columns
 - Provides APIs for defining, training, and optimizing models
- [ZenML](#)
 - Defines pipelines as composable steps and orchestrates execution
 - Pipelines can be extended by adding or reordering steps
 - Tracks pipeline state, artifacts, and execution metadata
- [Optuna](#)
 - Provides black-box hyperparameter optimization (e.g., TPE)
 - Isolates hyperparameter search code
 - Enables experimentation without modifying core implementations





Simulation Integration for Inference

- **Problem:** ML inferences are most efficient when batched on GPU; Gaudi processes event-by-event on CPU threads
 - **Solution #1:** Infer once in [pioneerML](#) (python) before Gaudi reconstruction
 - Pros: Efficient batching, independent development from simulations, simpler code development
 - Cons: Models cannot use reco data, requires additional I/O steps
 - **Solution #2:** Gaudi reco stages send data to a custom [Triton inference server](#), halt until response
 - Pros: Enables hot-swapping ML and traditional reconstruction stages, no I/O step
 - Cons: Efficient batching only emerges from many concurrent (10^2 to 10^3) jobs×threads (e.g. [AthenaTriton](#)), more complex code development
- Detector-only model(s) → **Solution #1 preferred**
 - Reco stages are informed by ML inferences in solution #1
 - Can still implement solution #2 if needed in the future



= Data in storage (SSD(s) or HDD(s))

= Data in system(s) RAM (CPU Tasks)

= Data in GPU(s) VRAM (GPU Tasks)

Inference Data Flow

- Same steps as training to stream/processes inputs
 - Caveat: Training targets are excluded
- Inference is best when batches saturate the GPU(s)
 - Inference batches are typically larger than training batches
 - Training: $O(10^1)$ graphs
 - Inference: $O(10^3)$ graphs
- Output construction is vectorized over the chunk
 - NumPy/PyTorch efficiently build output features (e.g. all hit particle classifications in an event) using precompiled C++
- Files are “finalized” once full
 - Output file structure mirrors input file structure

Training data is stored in a columnar format for efficiency. Currently use parquet files. RNTuple may also work.

Inference Input
Data Files

Data is streamed in chunks. This way we do not oversaturate RAM usage.

File Read
Streamer

Data is manipulated to form input tensors (e.g. construct graph nodes and edges tensors from data columns).

Input Adapter

Data is processed in batches on the GPU. Inference batch sizes are typically larger than training batch sizes.

Inference Batch

Output tensors are manipulated back into an event structure (e.g. take time group level predictions and attach them to the source event)

Output Adapter

Write *only* predictions to a file. Data streamed out of RAM into permanent storage in chunks.

File Write
Streamer

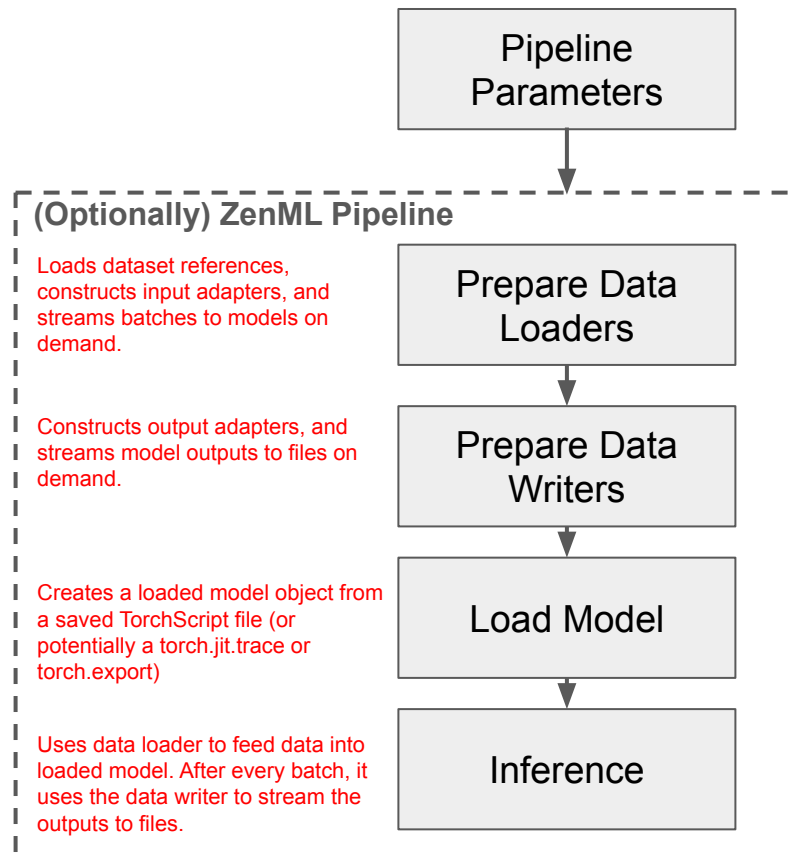
Prediction file rows are aligned with that of the input file (i.e. one row per event). Designed to “append” the original dataset

Prediction Files

Data Flow for Making Inferences

Model Inference Pipeline

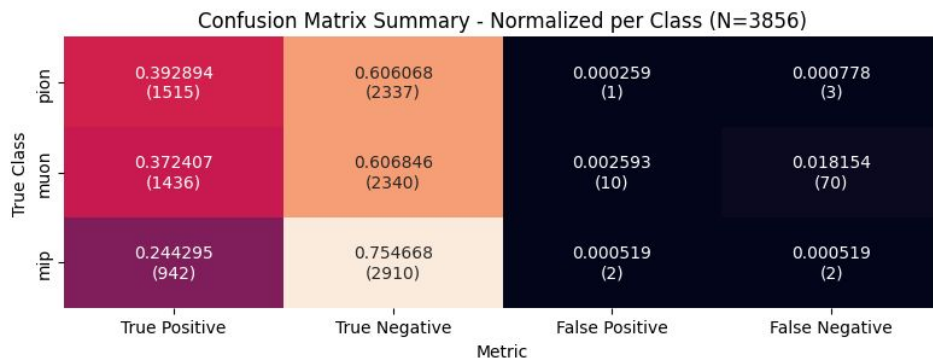
- [PyTorch](#)
 - Executes trained models using fast tensor operations
 - Performs forward passes to generate predictions
 - Supports exported models (TorchScript / torch.export)
- [ZenML](#)
 - Implemented as a ZenML pipeline to mirror training
 - Limited value for inference
 - Only the inference step performs computation
 - Other steps configure loaders, writers, and models
- Data Adapters
 - Stream events from storage to the model
 - Stream model predictions to storage
 - Currently implemented in Python
 - “solution #2” would benefit from migrating adapters to C++, potentially centralizing logic with pybinds



Simplified Example Pipeline for Training Models

Validation/Evaluation

- Validation: three main components, extensible via registered reusable implementations
 - Plots: produce figures
 - Metrics: produce non-plotted data
 - Evaluators: run a model and produce a set of plots/metrics from it's output
 - Meant to run in training pipeline
- Validation can also be ran without models
 - Compare truth datasets with prediction datasets
 - Use [polars](#) for dataset shaping
 - filtering, joins, feature derivation, etc.
 - Use custom plots or pre-registered plots to view extracted data
- [Example notebook](#)

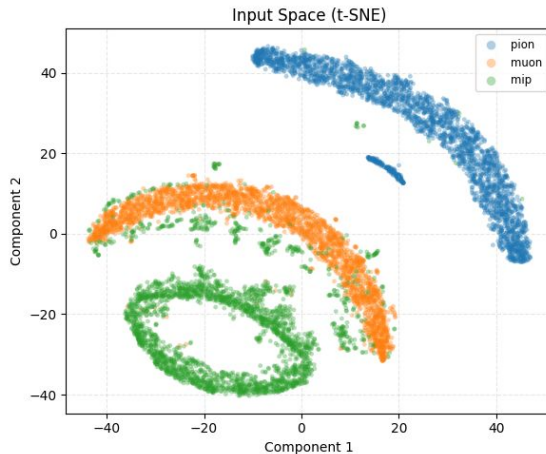


Example Validation Plot: “Confusion Matrix”
Normalized to Each Class

[TPE paper](#)

Validation Example (t-SNE)

- Can analyze embedding space to understand model behavior
 - [Example notebook](#)
- [t-distributed stochastic neighbor embedding](#) (t-SNE) is designed to preserve local clusters
 - Maps N dims $\rightarrow d$ dims
 - We choose $d = 2$ for visualization ease
- Other examples for these kinds of plots
 - [Uniform Manifold Approximation and Projection](#) (UMAP)
 - Often better at preserving global structure, but slower
 - [Principal Component Analysis](#) (PCA)

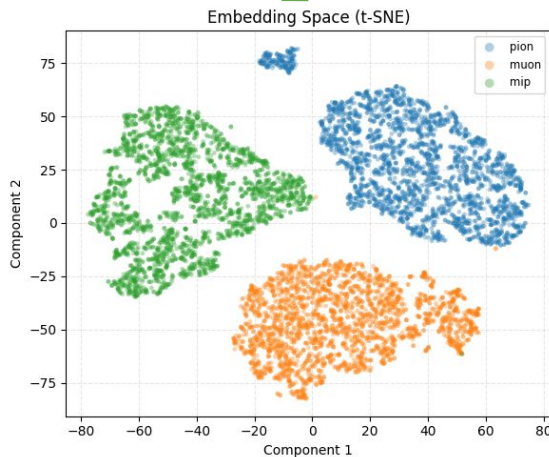


Average vector of all nodes in graph:
[coord,
z_pos,
energy,
view,
group_energy]

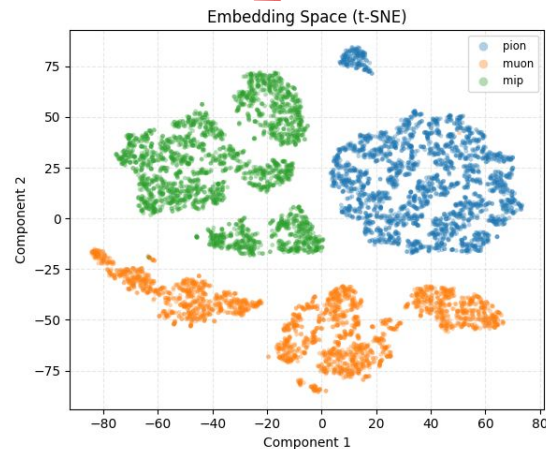
5D $\rightarrow$ 2D

Well-trained

Over-trained



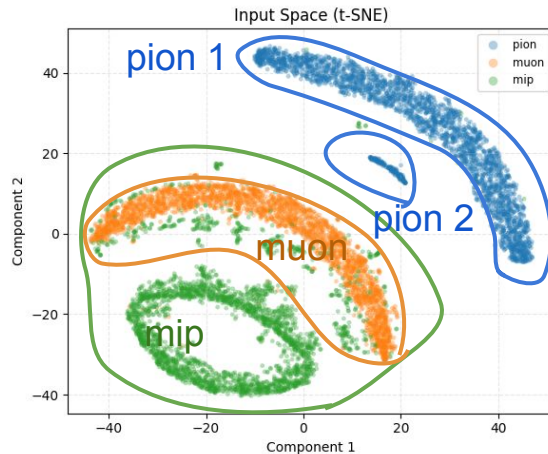
512D $\rightarrow$ 2D



512D $\rightarrow$ 2D

Validation Example (t-SNE)

- Can analyze embedding space to understand model behavior
 - [Example notebook](#)
- [t-distributed stochastic neighbor embedding](#) (t-SNE) is designed to preserve local clusters
 - Maps N dims $\rightarrow d$ dims
 - We choose $d = 2$ for visualization ease
- Other examples for these kinds of plots
 - [Uniform Manifold Approximation and Projection](#) (UMAP)
 - Often better at preserving global structure, but slower
 - [Principal Component Analysis](#) (PCA)

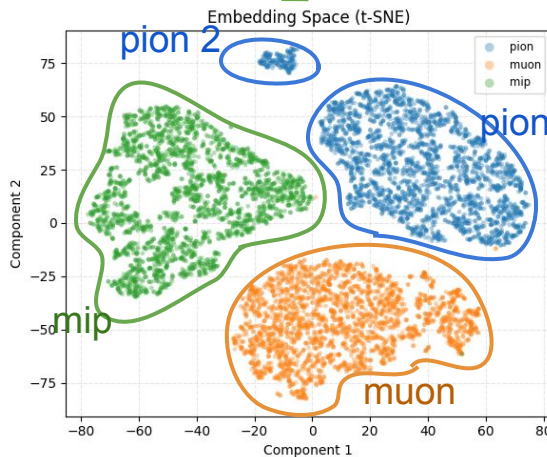


Average vector of all nodes in graph:
[coord,
z_pos,
energy,
view,
group_energy]

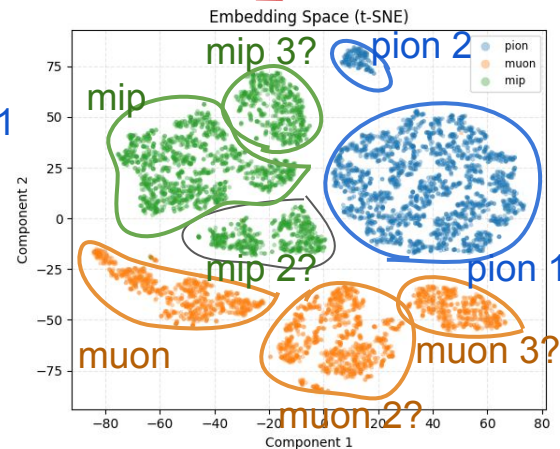
5D $\rightarrow$ 2D

Well-trained

Over-trained



512D $\rightarrow$ 2D



512D $\rightarrow$ 2D

Steps Before First ML Results with Simulation

- Develop the “unified model” (Omar) + develop pioneerML framework to handle scaled training + inference (Jack)
 - Framework in a functioning state but needs cleanup for integration of new models/inference strategies
- Integrate the unified model into the pioneerML repository
- Integrate ML ready dataset generator sub algorithm into the new v3 simulation
- Train the unified model on a dataset generated from the simulation framework
- Decide how inference will be done with the trained model
 - My suggestion:
 - Add pioneerML repo to the simulation framework
 - Add trained model to the simulation framework
 - Write a thin python Gaudi wrapper around the inference pipeline provided
 - This pipeline can be designed to run a Triton based inference to a server hosting a set of GPUs
- Run full simulation, compare ML inferences with traditional reco directly, etc.

Some Resources

- [pioneerML](#) framework (WIP, “beta”, functional with data from sim)
 - README has setup details
 - Docker (recommended)
 - Conda env
 - Contains [example notebooks](#) that shows how training, inference, and validation work at a surface level
 - Contains [tutorial notebooks](#) that show how to interact with core technology stack
- [Simulation setup](#) (WIP, but functional with some hacking)
 - Steps to add an ML training data writer to the simulation framework 2.1.0
 - Set-up should be streamlined in 2.1.1
 - TODO: migrate to v3, calo features, etc.

Auxiliary Slides

Training Data Loading

- The step itself creates a factory for making [pytorch DataLoaders](#)
 - A DataLoader provides batches for the model to train on
- General “streaming” sequence:
 1. Model requests batch
 2. Loader returns a prepared batch if available
 3. Otherwise, loader reads a file chunk
 4. Loader constructs many batches from a chunk
- Loader uses worker threads to prepare batches in the background while the model is training

Only select events that are in the training sample and are to be processed by this compute node

Join data split across multiple files (ex. Upstream predictions)

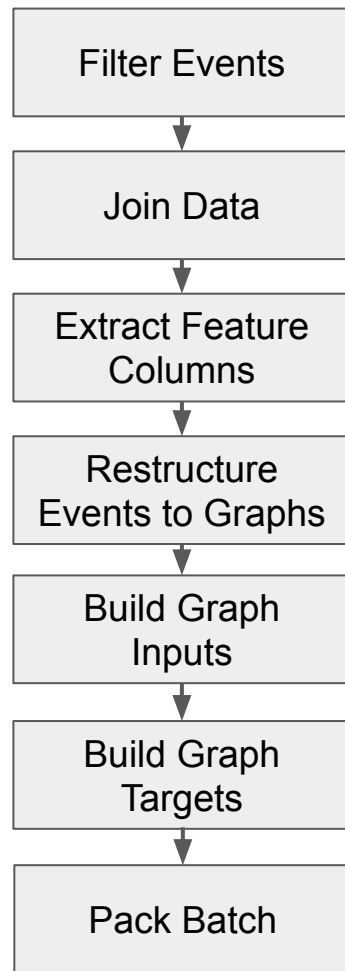
Only read in feature columns you want, ignoring all others.

Split events into graphs. Ex. one event may contain many time group graphs

Build nodes, edges, and global tensors efficiently from table data using tensorized operations

Build target tensors efficiently from table data using tensorized operations

Pack all data into one object that the model will receive

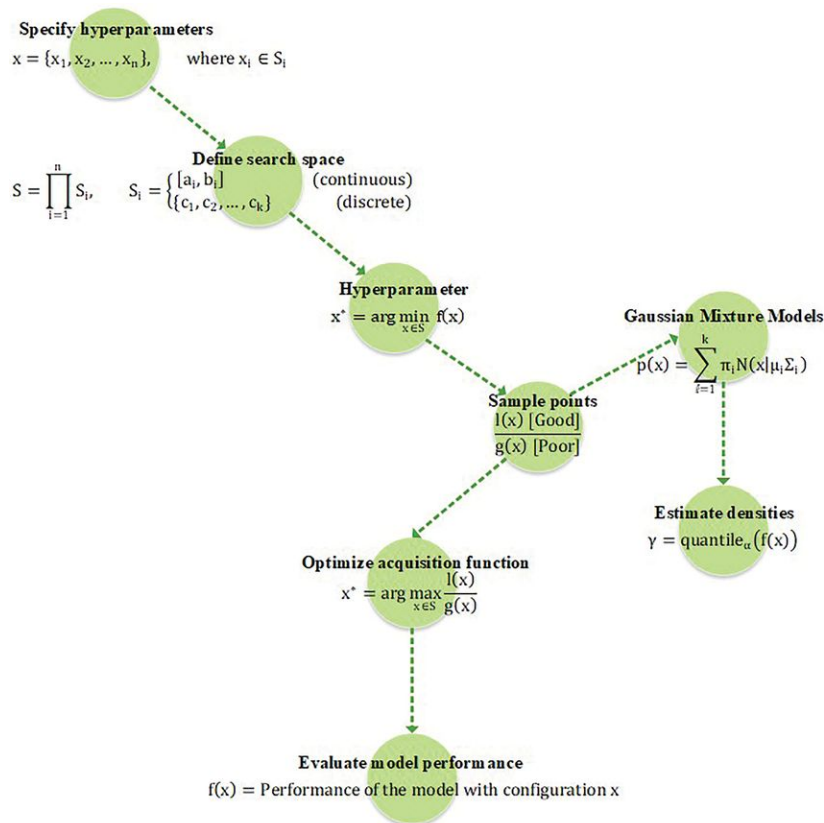


Example Graph Data Loading Sequence

Hyper Parameter Optimization

- Hyperparameter optimization (HPO) trains models across hyperparameter sets to minimize an objective (e.g. validation loss)
 - [Optuna](#) provides optimized black-box sampling algorithms for hyperparameter optimization
- Optuna supports many [sampling algorithms](#), examples:
 - [Tree-Structured Parzen Estimator](#) (TPE) [default]
 - Grid search
 - Random search
 - Gaussian process-based Bayesian optimization

Note: Uses the same training sequence (next slide) but with smaller samples across many trials

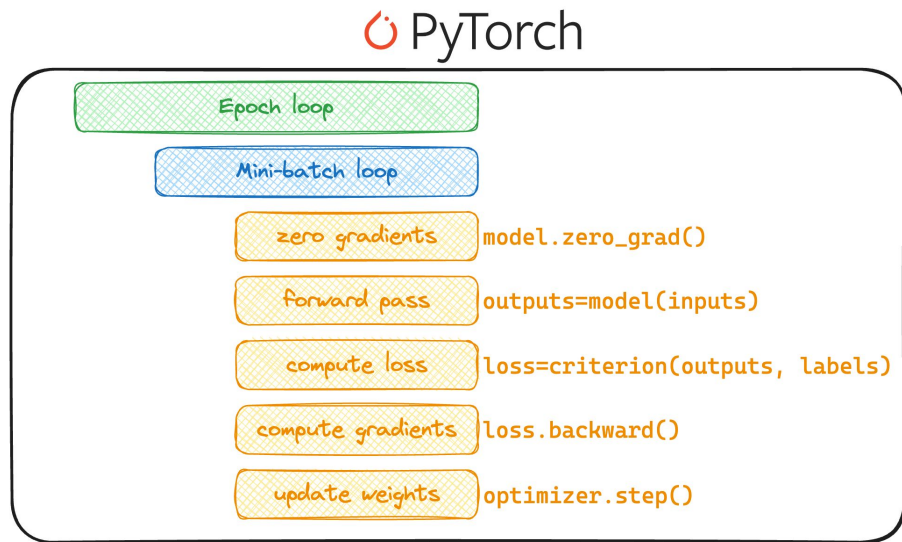


TPE flow diagram (in a nutshell)

[TPE paper](#)

Model Training

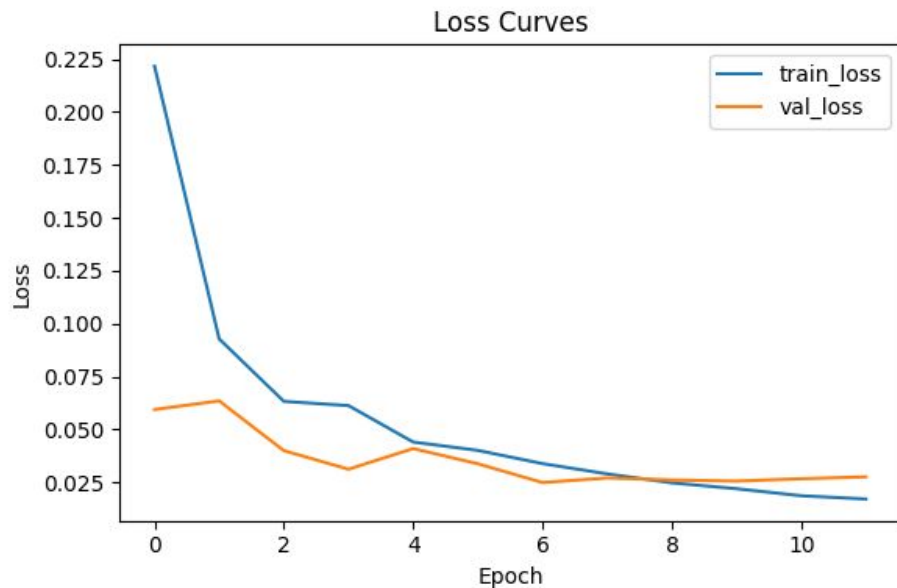
- Data is fed to a [lightning trainer](#) which handles:
 - Training/Validation loops
 - Backpropagation & optimizer steps
 - Checkpointing
 - Device management (CPU, GPU, etc.)
 - Distributed / multi-GPU training
- Optionally can [compile](#) modules for (potentially) faster training
- Optionally can provide [early stopping](#) parameters to prevent over training



Simplified training loop

Model Evaluation

- The trained model is tested on data not used during training
 - This data is the validation split of the full dataset
- Plots and metrics are stored in ZenML
 - Allows one to inspect a trained model's performance at a later date easily
- Example metrics:
 - Loss/accuracy history
 - Best loss/accuracy
 - Confusion matrix (for classifiers)



Example Loss + Validation Plot

Model Exporting

- Trained models are exported to files as [TorchScripts](#)
 - May migrate to [torch.export](#) if there are no plans for inference in C++
- Effectively saves the trained model
 - Additional benefit of being able to run the model across platforms, if needed

Note: Exportable models must follow certain constraints; not all Python operations are supported (e.g., dynamic loops or data structures)

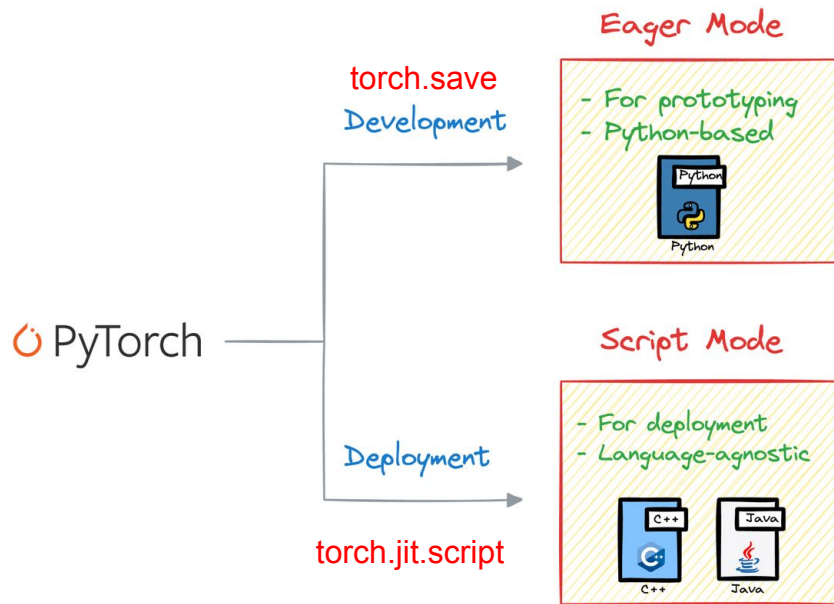


Illustration of Difference between different methods of saving models

Inference Data Loading

- Uses the same DataLoader as used for training but:
 - Doesn't look for training columns in the file
 - Doesn't build training target tensors
- Follows same streaming sequence, but uses bigger batches to saturate the GPU

Only select events that are in the training sample and are to be processed by this compute node

Filter Events

Join data split across multiple files (ex. Upstream predictions)

Join Data

Only read in feature columns you want, ignoring all others. Particularly, **no training features are extracted.**

Extract Feature Columns

Split events into graphs. Ex. one event may contain many time group graphs

Restructure Events to Graphs

Build nodes, edges, and global tensors efficiently from table data using tensorized operations

Build Graph Inputs

Build target tensors efficiently from table data using tensorized operations

Build Graph Targets

Pack all data into one object that the model will receive

Pack Batch

Example Graph Data Loading Sequence

Prediction Writing

- The step itself creates a factory for making a data writer
 - A data writer writes chunks of model predictions to files
- General “streaming” sequence:
 1. Model inference produces batch of predictions
 2. Writer receives the batch, validates it
 3. Writer stitches batch into a chunk of event level predictions
 4. Writer appends chunk to destination file
- Writer currently is single-threaded
 - May add multi-thread support if it becomes a bottleneck

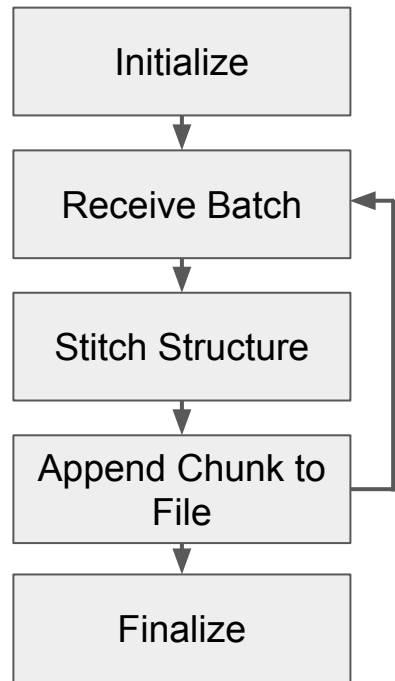
Prepare files, open write sinks

Receive output predictions, ensure they contain expected data payload

Restructure data to event style structure (e.g. take time group level predictions and attach them to the source event)

Add chunk of event level predictions to appropriate file, release memory.

Close all write sinks, finalize files with names and meta data.



Example Inference Data Writing Sequence

Model Loading

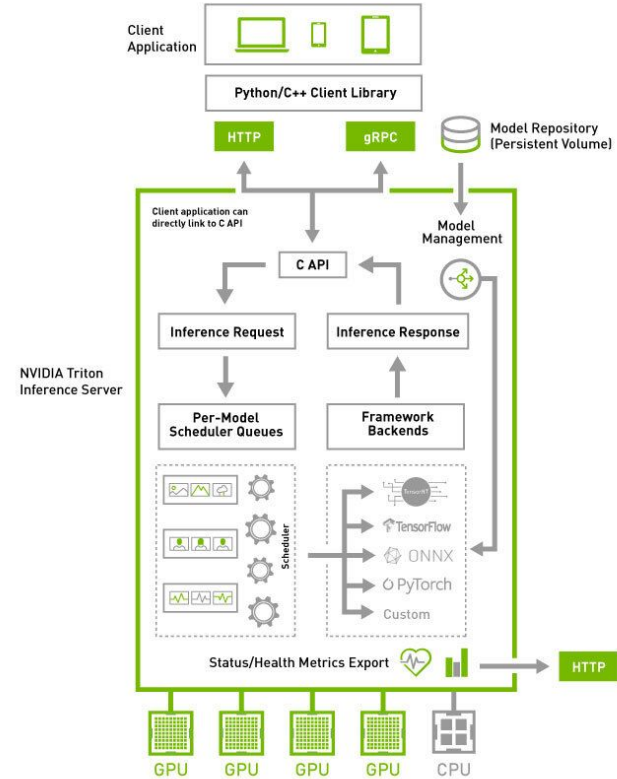
- Saved Torchscripts are loaded into a torch runtime
 - Could be python or C++ based
 - In our case, it's just python based
- TODO:
 - Add torch.export support, as TorchScripts have been deprecated

```
def load_torchscript(*, model_path: str, device: torch.device):  
    scripted = torch.jit.load(model_path, map_location=device)  
    scripted.eval()  
    return scripted
```

Torch provides simple methods for loading a model

Model Inference

- Trained models are streamed input batches
 - Model copies data to GPU
- Trained models process many samples (~many events) at once for efficiency
 - Best efficiency when GPU is saturated
- Trained model output prediction batches, which are then streamed to file
 - Copy predictions from CPU to GPU
- [NVIDIA Triton](#) allows data to be sent to a (optionally remote) server instead
 - Not currently integrated into pioneerML framework
 - Useful if machine you want inference data on does not have access to GPU(s)
 - Potentially useful for simulation workloads running on CPU clusters
 - Send data off to GPU machines instead and wait for a response



**NVIDIA Triton Server based Inference
Architecture for “Solution #2” Approach
[Not Currently Integrated]**

= Not an ML stage

= An ML (trained) stage

Current AI/ML ATAR Reconstruction Pipeline

Simulation Pipeline: Create ML Ready Dataset

Monte Carlo +
Detector Response

Truth level descriptions
of entire events +
detector response;
optional event mixing

ML Ready Dataset
Gaudi Pipeline

Forms ML ready dataset
(parquet files, efficient
columnar storage)
Applies preprocessing
stages to derive additional
feature columns (ex. Time
group labels)

AI/ML Pipeline: Append predictions to events, each stage follows the following prescription: Event Data → Graphs → Model → Graph Level Predictions → Event Level Predictions

Group Classifier

Adds group level
particle classes

Group Splitter

Adds hit level
particle classes

Endpoint Regressor

Adds endpoint (x,y,z)
estimates

Pattern Finder

Adds affinities between
time group

Pion Stop
Regressor

Adds pion stops
(x,y,z) estimates

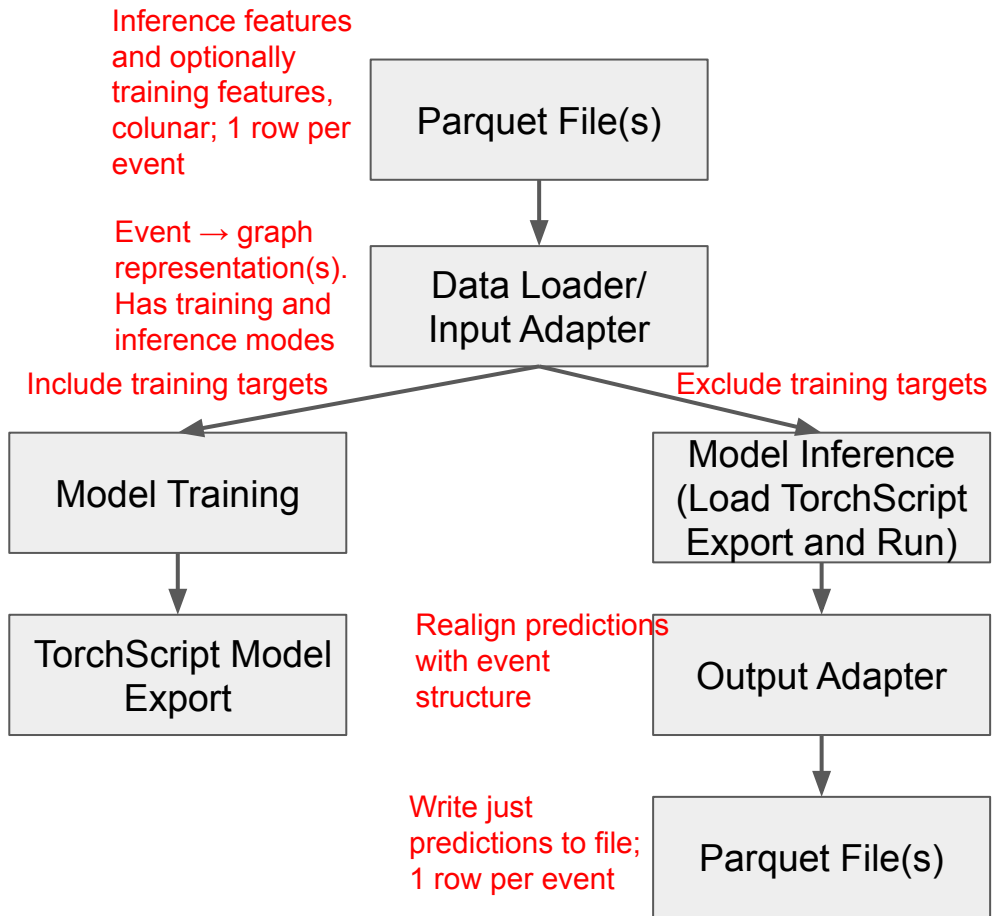
Positron Angle
Finder

Adds positron angles
($\sin\theta \cos\phi$, $\sin\theta \sin\phi$,
 $\cos\theta$) estimates

**Final Goal: Unify all stages into a single
end-to-end ML model**

Data Loading

- **Problem:** We will eventually scale to large training data sets
 - It becomes impossible to store all data in RAM at once
 - Operating one event at a time is too slow
 - Doesn't take advantage of parallel data loading operations
- **Solution:** Stream data into and out of models
 - Load data in one chunk at a time
 - Happens parallel with model execution
 - Model requests data when done training on it's current micobatch
 - Write data predictions in chunks as well



What is a Tree-Structured Parzen Estimator (TPE) (Part I)

- Given a (possibly stochastic) black box function you want to minimize (ex. model loss)

$$y = f(\theta)$$

- Hyperparams $\equiv \theta$
- Define the following:

$\gamma \equiv$ "good" / "bad" fraction (say, 0.2)

y^* is chosen such that $P(y < y^*) = \gamma$

so y^* is in, say, the best 20% of losses

$$\max(y^* - y, 0) \equiv \text{improvement}$$

- Then, for any given theta we can define *expected improvement*

$$\text{EI}(\theta) \equiv \mathbb{E}[\max(y^* - y, 0) \mid \theta] = \int_{-\infty}^{y^*} (y^* - y) p(y \mid \theta) dy$$

- Goal: Maximize expected improvement

What is a Tree-Structured Parzen Estimator (TPE) (Part II)

- For a 1 dimensional y , it turns out to be easier to work with $p(\theta | y)$, we can invert using Bayes' rule:

$$p(y | \theta) = \frac{p(\theta | y) p(y)}{p(\theta)}$$

- And substitute

$$\text{EI}(\theta) \propto \int_{-\infty}^{y^*} (y^* - y) \frac{p(\theta | y)}{p(\theta)} p(y) dy$$

- Since we don't know every value of y this becomes an impossible task. We must make an approximation by dividing into “good” and “bad” distributions

$$p(\theta | y) \approx \begin{cases} p(\theta | y < y^*) \equiv \ell(\theta), & y < y^* \text{ (good)} \\ p(\theta | y \geq y^*) \equiv g(\theta), & y \geq y^* \text{ (bad)} \end{cases}$$
$$p(\theta) \approx \ell(\theta) \int_{y < y^*} p(y) dy + g(\theta) \int_{y \geq y^*} p(y) dy = \gamma \ell(\theta) + (1 - \gamma) g(\theta)$$

What is a Tree-Structured Parzen Estimator (TPE) (Part III)

- The integral, by construction, only cares about the “good” region, so the integral simplifies to

$$\text{EI}(\theta) \propto \frac{\ell(\theta)}{p(\theta)} \int_{-\infty}^{y^*} (y^* - y) p(y) dy$$

- But the integral is now constant in theta! So we have:

$$\text{EI}(\theta) \propto \frac{\ell(\theta)}{p(\theta)}$$

- Where $p(\theta) = \gamma \ell(\theta) + (1 - \gamma) g(\theta)$ so we can write:

$$\text{EI}(\theta) \propto \frac{\ell(\theta)}{\gamma \ell(\theta) + (1 - \gamma) g(\theta)}$$

What is a Tree-Structured Parzen Estimator (TPE) (Part IV)

- But γ is fixed, so

$$\begin{aligned}\arg \max_{\theta} \text{EI}(\theta) &= \arg \max_{\theta} \frac{\ell(\theta)}{\gamma \ell(\theta) + (1 - \gamma) g(\theta)} \\ &= \arg \max_{\theta} \frac{\ell(\theta)/g(\theta)}{\gamma \ell(\theta)/g(\theta) + (1 - \gamma)} \\ &= \arg \max_{\theta} \frac{\ell(\theta)}{g(\theta)}\end{aligned}$$

- Where the final step is because $x/(bx+c)$ is monotonic in x for $x > 0$, let $x = \ell/g$
- In other words, we just need to find $\arg \max_{\theta} \frac{\ell(\theta)}{g(\theta)}$ which is doable via algorithm!

What is a Tree-Structured Parzen Estimator (TPE) (Part V)

- Now to define the algorithm, first we observe T (~ 10) trials randomly:

Observed trials: $\mathcal{D} = \{(\theta^{(i)}, y^{(i)})\}_{i=1}^T$

Quantile threshold: $P(y < y^*) = \gamma$

$$\mathcal{D}_{\text{good}} = \{\theta^{(i)} : y^{(i)} < y^*\}$$

$$\mathcal{D}_{\text{bad}} = \{\theta^{(i)} : y^{(i)} \geq y^*\}$$

- From this data, we want to build:

$$\ell(\theta) \equiv p(\theta \mid y < y^*)$$

$$g(\theta) \equiv p(\theta \mid y \geq y^*)$$

- So we define

$$\theta \equiv (\theta_1, \theta_2, \dots, \theta_d)$$

- And assume:

Independence assumption: $p(\theta \mid y < y^*) \approx \prod_{k=1}^d p(\theta_k \mid y < y^*)$

What is a Tree-Structured Parzen Estimator (TPE) (Part VI)

- This allows us to write:

$$\ell(\theta) \approx \prod_{k=1}^d \hat{p}_{\text{good}}(\theta_k)$$

$$g(\theta) \approx \prod_{k=1}^d \hat{p}_{\text{bad}}(\theta_k)$$

- We can fit to our samples to get a continuous distribution spaces for each param

$$\hat{p}_{\text{good}}(\theta_k) \leftarrow \text{fit to } \{\theta_k^{(i)} : \theta^{(i)} \in \mathcal{D}_{\text{good}}\}$$

$$\hat{p}_{\text{bad}}(\theta_k) \leftarrow \text{fit to } \{\theta_k^{(i)} : \theta^{(i)} \in \mathcal{D}_{\text{bad}}\}$$

- Then we sample from the “good” spaces for M candidates index by m

$$\tilde{\theta}_{m,k} \sim \hat{p}_{\text{good}}(\theta_k), \quad k = 1, \dots, d$$

- And finally, choose our next theta, add it to the data set, and repeat

$$\theta^{(t)} = \arg \max_{\tilde{\theta}_m} \frac{\ell(\tilde{\theta}_m)}{g(\tilde{\theta}_m)} = \prod_{k=1}^d \frac{\hat{p}_{\text{good}}(\tilde{\theta}_{m,k})}{\hat{p}_{\text{bad}}(\tilde{\theta}_{m,k})}$$

What is a Tree-Structured Parzen Estimator (TPE) (Part VII)

- How do we obtain our fits from the data?
- Uses [Kernel Density Estimation](#) (KDE)
 - The actual fit is done when determining each “gaussian kernel” (or sigma)
 - Likelihood-optimal is expensive $\sim O(n^2d)$
 - n = # samples
 - d = # hyperparameters
 - Optuna uses the “heuristic” version, which requires choosing some value for c .

Given samples: $\{\theta_k^{(i)}\}_{i=1}^n, \quad \theta_k^{(i)} \in \mathbb{R}$

Kernel density estimate:

$$\hat{p}(\theta_k) = \frac{1}{n} \sum_{i=1}^n \mathcal{K}\left(\frac{\theta_k - \theta_k^{(i)}}{h_k}\right)$$

Gaussian kernel:

$$\mathcal{K}(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$$

Bandwidth (likelihood-optimal):

$$h_k^* = \arg \max_h \sum_{i=1}^n \log \hat{p}_{-i}(\theta_k^{(i)} | h)$$

Bandwidth (heuristic used in practice):

$$h_k = c \cdot \text{std}\left(\{\theta_k^{(i)}\}_{i=1}^n\right), \quad c > 0$$